

1. Prinzipien

2. Praxis

- Aus- und Eingabe
- Fehler
- Entwicklung
- Testen
- Debuggen

Aus- und Eingabe

Ausgabe

Ausgabe auf Bildschirm über **Strom** (stream) **System.out**:

```
System.out.println(s);
```

schreibt Zeichenkette *s* auf den Bildschirm

nach Ausgabe **Zeilenumbruch**: folgende Ausgabe auf nächster Zeile

Variante **System.out.print** fügt **keinen Zeilenumbruch** an:

```
System.out.print("blumento");
```

```
System.out.println("pferde");
```

produziert Ausgabe "blumentopferde", dann Zeilenumbruch

folgende Zeilen sind äquivalent:

```
System.out.println(s);
```

```
System.out.print(s + "\n");
```

```
System.out.print(s + '\n');
```

```
System.out.print(s); System.out.println();
```

Ausgabestrom als Parameter

Typ von `System.out` ist `java.io.PrintStream`
(bietet Instanzmethoden `print` und `println`)

durch Übergabe als Parameter ist Trennung von **Inhalt**:

```
public static void gruesse(java.io.PrintStream ps,  
                           String name) {  
    ps.println(gruss(name));  
}
```

und **Ort** der Ausgabe möglich:

```
gruesse(System.out, "Welt");
```

schreibt "Hallo Welt!" auf den Bildschirm.

Eingabe

bisher: Werte zu Übersetzungszeit (als Literale, z.B. in main)

jetzt: **Eingabe** zu Laufzeit per Tastatur über **System.in**

Interpretation einer Eingabe hängt von erwarteten Typen ab

z.B. kann die Eingabe **16 kg** gelesen werden als

- 5 Zeichen '1' '6' ' ' 'k' 'g'
- Zeichenkette "16 kg"
- Zeichenketten "16" und "kg"
- Ganzzahl 16 und Zeichenkette "kg"
- Gleitkommazahl 16.0 und Zeichenkette "kg"
- ...

Klasse **java.util.Scanner** vereinfacht diese Interpretation:
explizite **Angabe von erwartetem Typ** eines einzugebenden Wertes

Eingabe: Beispiel

```
1 | import java.util.Scanner;           // Scanner-Klasse importieren
3 | public class Eingabe {
4 |     public static void echoSID() {
5 |         Scanner sc = new Scanner(System.in);
6 |                                     // Tastatur-Scanner oeffnen
7 |         String s = sc.next();        // Zeichenkette einlesen
8 |         int i     = sc.nextInt();    // Ganzzahl einlesen
9 |         double d = sc.nextDouble(); // Gleitkommazahl einlesen
10 |        System.out.println("gelesen: " + s + " " + i + " " + d);
11 |        sc.close();                  // Tastatur-Scanner schliessen
12 |                                    // (Ende Tastatur-Eingabe)
13 |    }
    // ...
60 | }
```

Zeilen 1 & 5 & 11 bilden **festes Verwendungsmuster** (später genauer)

Eingabe: Erläuterung

durch **Importieren** ist unqualifizierter Name **Scanner** verwendbar
(dazu später mehr)

Scanner liest jeweils geforderte Eingabe – **Fehler** falls unpassend
entsprechende Zeichen werden beim Lesen aus dem Strom entfernt

Eingaben werden **zeilenweise** verarbeitet, müssen also erst mit „Enter“
abgeschlossen werden (**Programmausführung wartet** bis dahin!)

Achtung: **Schließen** des Scanners schließt auch Strom – sollte

- in selber Methode geschehen wie das Öffnen
- für `System.in` erst am Ende des Programms (`main`) geschehen
(Beispiel also nicht gut!)

Eingabe: Beispiel separiert

da **Tastatur** **exklusive Ressource**, vorzugsweise Bereitstellung in main

```
1 import java.util.Scanner;           // Scanner-Klasse importieren
3 public class Eingabe {
14     public static void echoSIDpur(Scanner sc) {
15         String s = sc.next();        // Zeichenkette einlesen
16         int i = sc.nextInt();        // Ganzzahl einlesen
17         double d = sc.nextDouble(); // Gleitkommazahl einlesen
18         System.out.println("gelesen: " + s + " " + i + " " + d);
19     }
20     public static void main(String[] args) {
21         Scanner sc = new Scanner(System.in); // Tastatur-Eingabe
22         echoSIDpur(sc);
23         // ...                          // weitere Eingaben
24         sc.close();                     // Ende Tastatur-Eingabe
25         // ...                          // Code ohne Eingaben
26     }
60 }
```


Eingabe für Basistypen – genauer

`sc.next()`

- 1 überspringt führende White Space
- 2 erwartet/liest Folge von druckbaren Zeichen (kein White Space)
- 3 stoppt bei erstem White Space
- 4 liefert die Zeichenkette als Resultat

`sc.nextBoolean()` / `sc.nextInt()` / `sc.nextDouble()`

- 1 liest nächste druckbare Zeichenfolge per `sc.next()` – siehe oben
- 2 prüft, ob Zeichenkette gültigen Wert darstellt – sonst Fehler
- 3 wandelt die Zeichenkette in dargestellten Wert um
- 4 liefert den Wert als Resultat

Eingabe für Basistypen

Scanner-Instanzmethoden zur **Eingabe von Basistyp-Werten**:

Instanzmethode	Ergebnistyp	Eingabe
<code>next()</code>	String	druckbare Zeichen vor White Space
<code>nextBoolean()</code>	boolean	true oder false
<code>nextInt()</code>	int	Ganzzahl
<code>nextDouble()</code>	double	Gleitkommazahl

Hinweis: in **deutscher Systemumgebung** wird Gleitkommazahl mit **Komma** statt **Dezimalpunkt** gelesen (und intern umgewandelt)

White Space vor der Eingabe wird **nicht ignoriert** von:

Instanzmethode	Ergebnistyp	Eingabe
<code>nextLine()</code>	String	alle Zeichen (incl. White Space) bis (excl.) Zeilenende

Eingabe bis Ende

Scanner-Instanzmethode **hasNext()** liefert Wahrheitswert, ob noch eine Zeichenkette (außer White Space) folgt:

```
27 | public static void echo(Scanner sc) {  
28 |     while (sc.hasNext()) {  
29 |         System.out.println(sc.next());  
30 |     }  
31 | }
```

Eingabe.java

analog: **hasNextBoolean()** / **hasNextInt()** / **hasNextDouble()**
geben an, ob entsprechende Eingabe folgt: **false**, wenn:

- **unpassende** Eingabe folgt (nicht möglich für **hasNext()!**)
- **Eingabeende** erreicht

Eingabeende

explizites Beenden der Eingabe durch:

- POSIX: **Ctrl-d** bzw. **Strg-d**
- MS CMD: **Ctrl-z** bzw. **Strg-z** (gefolgt von **Enter**)

am Zeilenanfang (Eingaben werden zeilenweise verarbeitet)

Hinweis: „Markierung“, kein Zeichen

entspricht Dateiende (**EOF**: „end of file“) bei Lesen aus Datei

Aus- und Eingabeumlenkung (systemseitig)

Ausgabeumlenkung > aus Standardausgabe in Datei

```
$ java Programm > Datei
```

→ keine Ausgabe auf dem Bildschirm!

Eingabeumlenkung < aus Datei (bis **Dateiende**) in Standardeingabe

```
$ java Programm < Datei
```

→ keine Eingabe von der Tastatur!

kombinierbar:

```
$ java Programm < DateiEin > DateiAus
```

praktisch für:

- **umfangreiche** Ein-/Ausgaben
- **Wiederholung** von Eingaben
- **Vergleichen** von Ausgaben zu Testzwecken

Beispiel: Eingabeende für bestimmten Wert

Wiedergabe bis Eingabeende oder bis s gelesen wurde:

```
32 public static void echoBis(Scanner sc, String s) {  
34     while (sc.hasNext()) {  
35         String n = sc.next();  
36         if (n.equals(s)) {  
37             return;  
38         }  
39         System.out.println(n);  
40     }  
46 }
```

→ alternativ Anwendungsbeispiel für **konditionale Auswertung**:

```
42 String n;  
43 while (sc.hasNext() && !(n = sc.next()).equals(s)) {  
44     System.out.println(n);  
45 }
```

Übung: Eingabe.zaehlen

Aufgabenblatt 03, Aufgabe 03

Beispiel: älteste eingegebene Person

```
1 | import java.util.Scanner;           // Scanner-Klasse importieren
   // ...
47 | public static String aeltester(Scanner sc) {
48 |     String maxName = "";
49 |     int maxAlter = -1;    // hier o.k. (warum?)
50 |     while (sc.hasNext()) {
51 |         String n = sc.next();
52 |         int a = sc.nextInt();
53 |         if (maxAlter < a) {
54 |             maxName = n;
55 |             maxAlter = a;
56 |         }
57 |     }
58 |     return maxName;
59 | }
```


Übung: Eingabe.durchschnitt

Aufgabenblatt 03, Aufgabe 04

Eingabe: Beispiel „BS-Bingo“

```
1 import java.util.Scanner;
2 public class BsBingo {
3     public static void play(Scanner sc, String buzz) {
4         int count = 0;
5         while (sc.hasNext()) {
6             String s = sc.next();
7             if (s.equals(buzz)) {
8                 ++count;
9                 if (count == 3) {
10                     System.out.println("Bingo!");
11                     return;
12                 }
13             }
14         }
15     }
16 }
```

Übung: Messdaten.durchschnitt

Aufgabenblatt 03, Aufgabe 05

Fehler

Übersetzungsfehler

Übersetzungsfehler werden gemeldet für Fehler, die zur Übersetzungszeit (also **unabhängig von Daten**) erkannt werden:

- **Syntaxfehler**: Abweichung von der vorgeschriebenen Struktur (falsche Schreibweise oder Reihenfolge, fehlende Elemente, ...)
- **Semantikfehler** ~ **Typfehler**:
 - Verwendung nicht deklariertem (Methoden-/Variablen-/...) Bezeichner, also von unbekanntem Typ
 - Verwendung von Wert entspricht nicht seinem deklarierten Typ

Laufzeitfehler: Beispiel

datenbezogene Fehler werden bei Laufzeit entdeckt und gemeldet:

```
1 public class Wohnraum {
2     public static int flaecheMittel(int flaecheGesamt,
3                                     int anzahlWohnung) {
4         return flaecheGesamt / anzahlWohnung;
5     }
6 }
```



```
1 public class WohnraumTest {
2     public static void main(String[] args) {
3         // Aufruf mit 0 Wohnungen unsinnig
4         System.out.println(Wohnraum.flaecheMittel(640, 0));
5     }
6 }
```

(0 ist hier zwar Literal, könnte aber auch aus Eingabe stammen)

Laufzeitfehler: Konsequenz

Laufzeitfehler entstehen bei Ausführung für **ungültige Werte**:
Meldung mittels **Exception** und (vorerst) **Programmabbruch**

```
$ java WohnraumTest
Exception in thread "main"
    java.lang.ArithmeticException: / by zero
    at Wohnraum.flaecheMittel(Wohnraum.java:4)
    at WohnraumTest.main(WohnraumTest.java:4)
$
```

Stack Trace zeigt Stellen der Aufrufe, die zum Fehler führten

Entwicklung

Entwicklung

Frage: wie geht man vor, eine gegebene Prog.aufgabe zu lösen?

Hinweis: allgemeiner **Prozess** Thema von „Software Engineering“;
hier Fokus auf „kleine“ Aufgaben

Ziele:

- **notwendig: funktionale Anforderungen** erfüllen:
Code liefert **korrekte Ergebnisse für alle Eingaben**
- (vorerst nur) wünschenswert:
nichtfunktionale/qualitative Anforderungen erfüllen:
 - **Effizienz** (Speicherbedarf, Laufzeit)
 - **Codequalität** (Übersichtlichkeit, Klarheit, Dokumentation)
 - ...

Entwicklung: Vorgehen („Hands-on“-Fassung)

- ➊ Aufgabe ganz lesen und verstehen; Wichtiges markieren
- ➋ Beispiele erstellen, mit Aufgabe abgleichen (verstanden?)
- ➌ Beispiele in ein Testprogramm übertragen
- ➍ Skizze für (relevante) Beispiele erstellen
- ➎ Vorgehensweise überlegen, in Skizze/Stichworten umsetzen
- ➏ Skizze mit Aufgabe abgleichen; Beispiele gedanklich durchspielen
- ➐ Code „von außen nach innen“ erstellen und dokumentieren
- ➑ Code mit Skizze abgleichen; Beispiele gedanklich durchspielen
- ➒ Code übersetzen, ggf. Fehler suchen
- ➓ Testprogramm ausführen, ggf. Fehler suchen
- ➔ Code ggf. von Team begutachten und testen lassen
- ➕ Code von Auftraggeber abnehmen lassen

bei Fehler: Wiederholung ab Ursprung(!) des Fehlers

Entwicklung: Effizienz und Algorithmen

auch einige **nichtfunktionale Eigenschaften** müssen im Entwurf **von Anfang an** berücksichtigt werden:

Effizienz abhängig von Wahl des **Algorithmus** (Berechnungsweg)

Beispiel: Summenberechnung per Rekursion/Schleife unsinnig!

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n \cdot (n + 1)}{2}$$

damit Berechnung durch nur 3 statt $\sim n$ Operationen:

```
2 | public static int summe(int n) {  
3 |     return n * (n + 1) / 2;  
4 | }
```

Mathe2.java

Entwicklung: wichtige Entwurfsprinzipien

anschaulich: „Bausteinprinzip“

- Modularisierung
- Separierung / Entkoppelung
- Abstraktion
- Spezifikation / Exaktheit
- Einfachheit („so einfach wie möglich, aber nicht einfacher“)
- Klarheit
- Wiederverwendbarkeit
- Erweiterbarkeit

→ Abhängigkeiten, Wiederholungen, Überflüssiges vermeiden

Testen

Testen

Wie prüft man, ob ein Programm(teil) **korrekt** funktioniert?
„Ansehen“ genügt nur bei kleinen Programmen und viel Erfahrung.

Testen besteht aus der Anwendung unterschiedlicher **Testfälle**:

- ➊ Auswahl von **Testdaten**
- ➋ **Ausführen** des Programms oder der Methode für diese Daten
- ➌ Beobachtung von **Verhalten** und **Resultat**
- ➍ **Vergleich** mit **Erwartung** laut **Spezifikation** des Programms

Weil **vollständige** Abdeckung i.d.R. **nicht möglich** ist:

Testen gibt **keine Garantie**, nur „**wahrscheinlichere**“ **Korrektheit**!
(**negative Aussage**: „hier Fehler“ – i.d.R. keine positive Aussage)

Testebenen

verschiedene **Testebenen** testen Code in unterschiedlicher Auflösung:

- **Unit-Test/Komponententest**: einzelne **Klasse/Methode**
(Argumente vs. Ergebnis als Testdaten)
- **Integrationstest**: Zusammenspiel mehrerer **Klassen/Methoden**
- **Systemtest**: **Programm**, aus Perspektive der Nutzenden
(Eingabe vs. Ausgabe als Testdaten)
- **Akzeptanztest**: **Programm**, aus Perspektive der Auftraggeber

Hinweise:

- hier (Kleinprojekte) fallen die Ebenen meist zusammen
⇒ Fokus auf **Unit-Tests**
- Testen ist **umfangreiches Thema** → SE I+II

Unit-Test: Beispiel (naiv)

intuitives Vorgehen: „Ausprobieren“

```
1 public class MatheTestNaiv {  
2     public static void main(String[] args) {  
3         System.out.println(Mathe.max(5, 3));  
4         System.out.println(Mathe.max(-2, 316));  
5         // ...  
6     }  
7 }
```

Nachteile:

- willkürliche Auswahl, wenig zuverlässig
 - erfordert „manuellen“ Vergleich von Testcode und Ausgaben
- ⇒ aufwendig und fehleranfällig

Unit-Teststrategie

Teststrategie: möglichst viele **kritische Fälle** abdecken

dazu betrachtet man z.B.:

- typabhängige **Sonder-**(z.B. **Extrem-**)**werte**
(0, 1, -1, min./max. Zahlwert, leere Zeichenkette, ...)
- **problemabhängige** spezielle **Werte/-kombinationen**
- **Zufallswerte** im Wertebereich

Ziel: möglichst viele/alle **Varianten der Ausführung** abdecken

Unit-Teststrategie: Beispiel

Aufgabe: Methode `max` liefert Maximum von 2 ganzen Zahlen

mögliche Probleme (und Testfälle, sie aufzudecken):

- Lösung kann nicht mit 0 oder mit negativen Zahlen umgehen

`max(0,1) == 1`

`max(0,-1) == 0`

`max(-1,-2) == -1`

- Lösung liefert falschen Wert, wenn die Zahlen gleich sind

`max(1,1) == 1`

`max(-5,-5) == -5`

`max(0,0) == 0`

- Ergebnis der Lösung ist abhängig von Reihenfolge der Werte

`max(1,2) == 2`

`max(2,1) == 2`

- ...

Unit-Test: Beispiel (umständlich) I

```
1 public class MatheTestLang {
2     public static boolean test() {
3         boolean resultat = true;
4         if (Mathe.max(0,-1) != 0) {
5             System.out.println("FEHLER: max(0,-1)=="
6                               + Mathe.max(0,-1) + " statt 0");
7             resultat = false;
8         }
9         if (Mathe.max(-1,-2) != -1) {
10            System.out.println("FEHLER: max(-1,-2)=="
11                              + Mathe.max(-1,-2) + " statt -1");
12            resultat = false;
13        }
14        if (Mathe.max(1,1) != 1) {
15            System.out.println("FEHLER: max(1,1)=="
16                              + Mathe.max(1,1) + " statt 1");
17            resultat = false;
18        }
19    }
20 }
```

Unit-Test: Beispiel (umständlich) II

```
19  if (Mathe.max(-29,18) != 18) {
20      System.out.println("FEHLER: max(-29,18)==\"
21                          + Mathe.max(-29,18) + \" statt 18)");
22      resultat = false;
23  }
24  // ...
25  return resultat;
26  }
27  public static void main(String[] args) {
28      if (test()) {
29          System.out.println("OK");
30      }
31  }
32 }
```

Problem: viele **Wiederholungen** → verstößt gegen **Wiederverwendung**

Unit-Test: Beispiel (smarter)

```
1 public class MatheTest {  
2     public static boolean maxCheck(int a, int b, int erw) {  
3         int erg = Mathe.max(a, b);  
4         boolean istKorrekt = (erg == erw);  
5         if (!istKorrekt) {  
6             System.out.println("FEHLER: max(" + a + ", " + b + ")=="  
7                 + erg + " statt " + erw);  
8         }  
9         return istKorrekt;  
10    }
```

Klassenmethode zum Prüfen eines max-Aufrufs, z.B. durch Aufruf:

```
maxCheck(0, 0, 0); // liefert true/false
```

```
11 public static boolean maxTest() {
12     return maxCheck(0, 0, 0)
13         & maxCheck(-1, 1, 1)
14         & maxCheck(1, -1, 1)
15         & maxCheck(-29, 18, 18)
16     /*& ... */;
17 }
18 // ggf. Tests weiterer Methoden, z.B. 'absTest()'
19 public static boolean test() {
20     return maxTest()
21         /*& absTest()*/
22         /*& ... */;
23 }
24 public static void main(String[] args) {
25     if (test()) {
26         System.out.println("OK");
27     }
28 }
29 }
```

Unit-Test: Beispiel (smarter) erläutert

Verknüpfung einzelner Tests mit dem Operator `&` z.B. in

```
20 |     return maxTest()  
21 |         /*& absTest()*/  
22 |         /*& ...      */;
```

bewirkt Ausführung **aller** Tests – mit Operator `&&` endet das Testprogramm beim **ersten** fehlgeschlagenen Test

Hinweis: Verhalten von `&&` besser bei großen Projekten (viele Tests) erfordert Sortierung nach Abhängigkeit: elementare Methoden zuerst

Übung: Datum.istSchaltjahr Test

Aufgabenblatt 04, Aufgabe 01

Debuggen

Fehlersuche: Kontrollausgaben

naives Vorgehen für Fehlersuche / besseres Codeverständnis:

- ➊ Ausgabeanweisungen (`System.out.println(...)`) an „interessanten“ Stellen einfügen
- ➋ darin „interessante“ Werte ausgeben
- ➌ kompilieren, ausführen, Werte mit Code vergleichen
- ➍ ab 1. wiederholen, auf diese Weise zum Fehler vorarbeiten
- ➎ nach Beheben des Fehlers Ausgaben wieder entfernen

⇒ **unsystematisch** (was ist „interessant“?), **umständlich**, **anfällig**

→ Änderungen selbst können anderes (richtiges/falsches) Verhalten verursachen und dadurch in die Irre führen

Fehlersuche: Debugger

ein **Debugger** ist ein Werkzeug, das erlaubt,

- ein Programm **schrittweise** auszuführen
- dies zugleich **am Quellcode** nachzuvollziehen
- Werte von **Variablen anzuzeigen** (und sogar zu ändern)
- Werte für **Ausdrücke anzuzeigen**

und so den **Ablauf genau nachvollziehen** zu können

Neukompilieren ist nicht notwendig; der Code bleibt unverändert

aber: **Debuggersteuerung** in der **Kommandozeile** ist **kompliziert**

IDE

IDE (**I**ntegrated **D**evelopment **E**nvironment):

(meist grafische) integrierte Entwicklungsumgebung

bietet: Codeerzeugung, -umgestaltung, -markierung, -analyse, ...

insbesondere: vereinfachte Benutzung des Debuggers

gut:

- kann Produktivität erfahrener Entwickler sehr steigern

schlecht:

- ist komplex, Konfiguration/Verständnis erfordern Kenntnisse
- übernimmt Aufgaben, bremst Lernfortschritt („Stützradeneffekt“)
- kann Anfänger verwirren (Hinweise/erzeugter Code unpassend)

Hinweis: zur Übung Fehleranzeige, Codeergänzung etc. deaktivieren

Beispiele: Eclipse, NetBeans, IntelliJ, JDeveloper, BlueJ, ...

Debugger: Arbeitsweise

typische Verwendung des Debuggers:

- ➊ **Breakpoint** setzen, ab wo untersucht werden soll
- ➋ Programmstart („Run“) → hält bei Breakpoint
- ➌ **schrittweises** Durchlaufen:
 - „step over“: aktuelle **Zeile ausführen**
 - „step into“: **in nächsten Methodenaufruf** gehen (Sprung)oder **Weiterlaufen** bis zu bestimmter Stelle:
 - „step return“: **nächstes return** (Rücksprung)
 - „continue“: **nächster Breakpoint**
- ➍ **Anzeigen/Verfolgen** des aktuellen Status:
 - **Parameter** und **Variablen**
 - **Ergebnis** von Methodenaufruf (nach „step return“)
 - **Ausdrücke** auswerten

viele weitere Möglichkeiten (siehe Dokumentation)

Übung: Debugger Mathe.istPrimzahl

Aufgabenblatt 04, Aufgabe 02

Übung: Debug Mathe.min/mean

Aufgabenblatt 04, Aufgabe 03

Übung: Debug DatumBug

Aufgabenblatt 04, Aufgabe 04